scai

SORBONNE CLUSTER FOR
ARTIFICIAL INTELLIGENCE

Quiz

1. Quelle est la différence entre un neurone artificiel et la régression linéaire, logistique?
2. Qu'est ce qui compose un réseau de neurone?
3. Combien y a-t il de paramètre dans un neurone ? Comment initie-t-on les poids?
4. Comment entraîne-t-on un NN?
5. A quoi sert l'early stopping?

Course 5 DS — From Linear Model To Large Language Model

Raphael Cousin
Data Scientist
raphael.cousin@sorbonne-universite.fr

raphaelcousin.com

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Plan

From Linear to Non-Linear

Duration : ~10 min

Multi-Layer Perceptron

Duration : ~20 min

Architectures

Duration : ~10 min

Training

Duration : ~10 min

Practical Considerations

Duration : ~10 min

MLP in scikit-learn

Duration : ~10 min

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

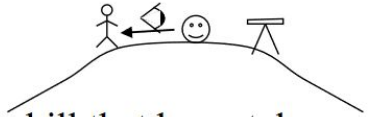
Natural Language Data

Sequential Nature of Text

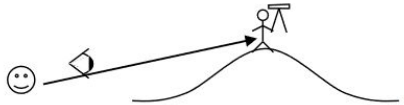
Word order impacts meaning: "Dog bites man" ≠ "Man bites dog"

Dependencies can span across **long distances in a sequence**

Context is critical for disambiguation (e.g., "bank" can mean financial institution or river edge)

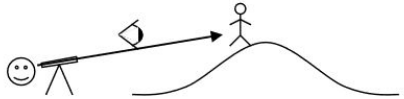


"I was on the hill that has a telescope when I saw a man."

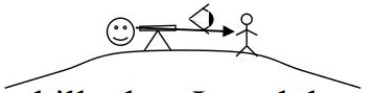


"I saw a man who was on a hill and who had a telescope."

"I saw a man who was on the hill that has a telescope on it."



"Using a telescope, I saw a man who was on a hill."



"I was on the hill when I used the telescope to see a man."

. . .

I saw the man on the hill with the telescope

Language Diversity at Scale

Languages and Writing Systems

~7,000 living languages worldwide (Ethnologue 2023)

~300 writing systems across history, with ~150 currently in use

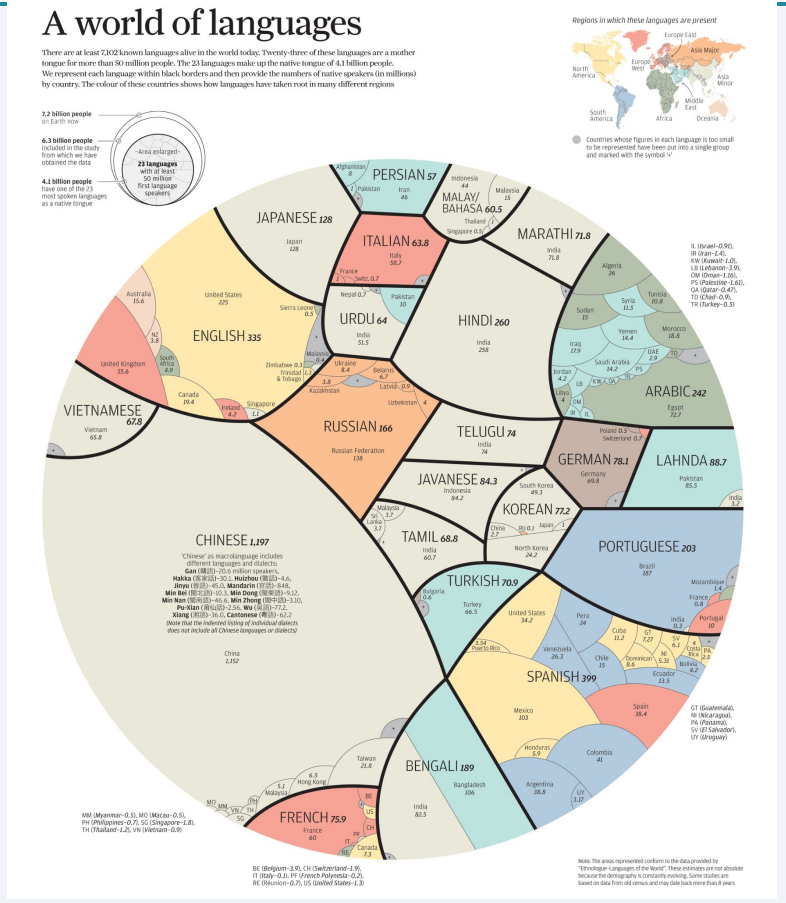
Vocabulary and Text Volume

170,000+ words in current English usage (Oxford English Dictionary)

~130 million books published in all languages throughout history

2.5 million+ books published annually worldwide

Billions of web pages containing trillions of words across languages



scai SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Sequence Representations Beyond Natural Language

Biological Sequences: DNA sequences use four nucleotide bases: A (Adenine), T (Thymine), G (Guanine), C (Cytosine).

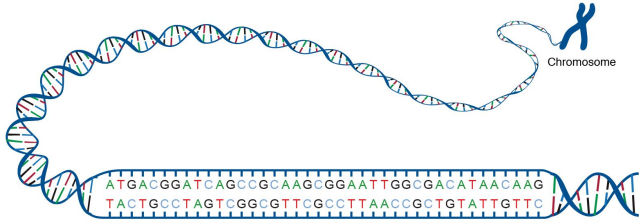
Musical Sequences: MIDI note numbers (0-127), pitch classes (C, C#, D, D#, E, F, F#, G, G#, A, A#, B), duration values (whole, half, quarter notes)

$$\int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}$$

$$f(x) = a_0 + \sum_{n=1}^{\infty}\left(a_n \cos\frac{n\pi x}{L} + b_n \sin\frac{n\pi x}{L}\right)$$

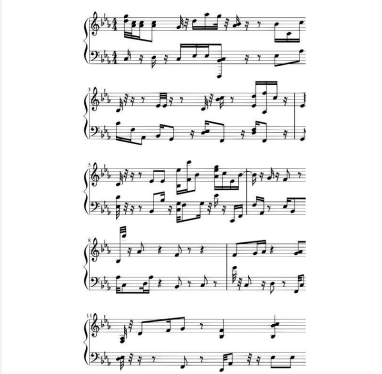
$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Programming Code: Programming languages like Python, Java, and C++ have finite token vocabularies including keywords (if, while, return), operators (=, +, ==), and punctuation (braces, parentheses, semicolons).



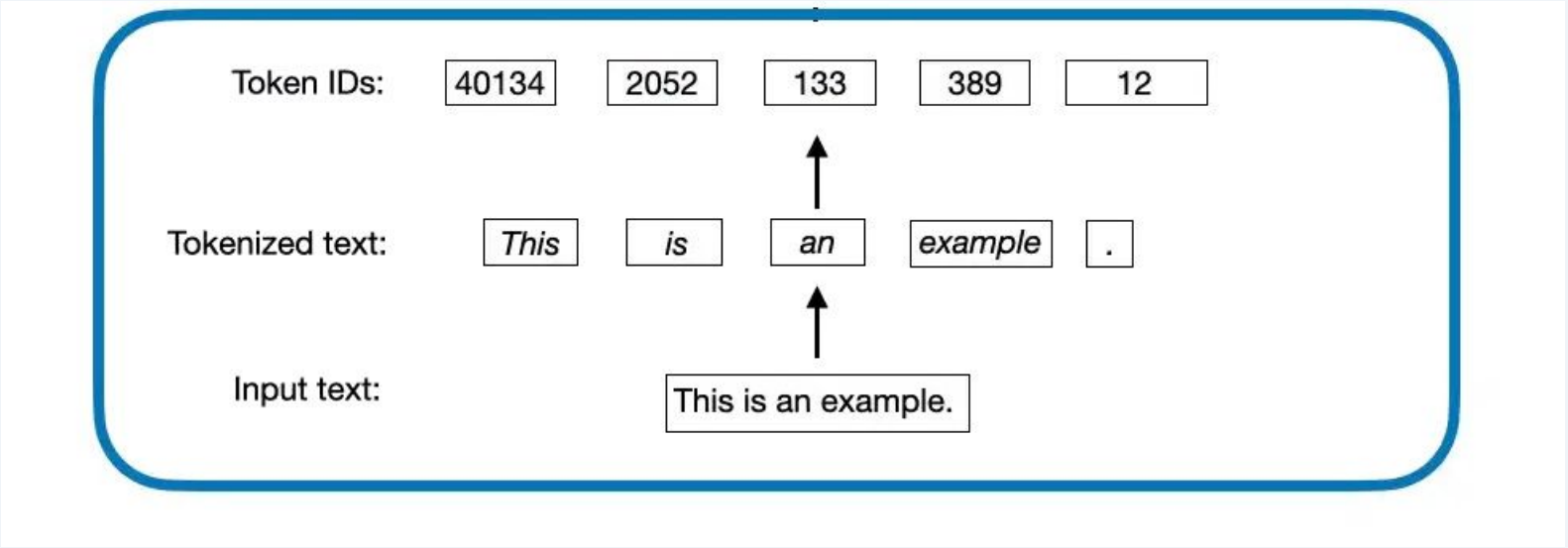
Chemical Representations: SMILES (Simplified Molecular Input Line Entry System) notation represents molecular structures as text strings using characters for atoms (C, N, O, S), bonds (-, =, #), and branches (parentheses)

Mathematical: Mathematical notation forms a symbolic language with operators (+, -, ×, ÷, =), variables (x, y, z), functions (sin, cos, log), Greek letters (α, β, γ), and special symbols (∫, ∑, ∂)



Tokenization - Represent text as numerical values

Tokenization



Tokenization = The process of breaking text into smaller units (tokens) that serve as the basic elements for numerical representation.

Vocabulary = A fixed collection of all possible tokens that the tokenizer recognizes.

Character-Level Tokenization

Vocabulary: Alphabet + special characters (~30 tokens)

Character-Level Tokenization

Character tokenization breaks text into individual characters, offering a very small vocabulary but requiring longer sequences.

Example :

Token	ID	Token	ID
"a"	1	"z"	26
"b"	2	" "	27
"c"	3	"."	28

vocabulary

"The cats are running" → [T,h,e, ,c,a,t,s, ,a,r,e, ,r,u,n,n,i,n,g] → [20,8,5,27,3,1,20,19,27,1,18,5,27,18,21,14,14,9,14,7]

❌ **Problems:** Very long sequences + loss of word structure

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Word-Level Tokenization

Vocabulary: All the words in the corpus (example: English Wikipedia ~13M tokens)

Word-Level Tokenization

Word tokenization splits text at word boundaries, typically using spaces and punctuation as delimiters.

Example :

Token	ID
"The"	1
"cats"	856,432
"running"	2,341,567

vocabulary

"The cats are running" → [The, cats, are, running] → [1, 856432, 15, 2347]

❌ **Problems:** Gigantic vocabulary + rare words underrepresented + "eat" ≠ "eats"

SubWord - Level Tokenization

Byte-Pair Encoding

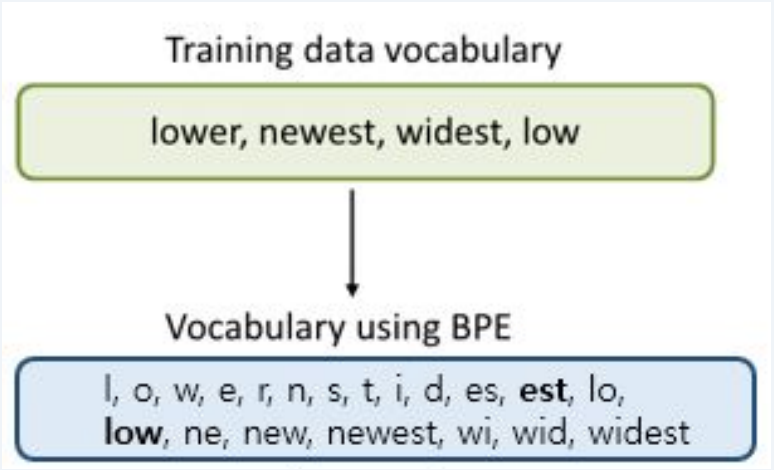
BPE iteratively merges the most frequent adjacent byte pairs or character pairs to form new subword tokens.

1. Start with **character level vocabulary**
2. Identify the **most frequent pair** of adjacent tokens
3. **Add it to the vocabulary**
4. Iterate and stop at ~10k-100k tokens

Example :

Token	ID	Fréquence
"a"	1	
"s"	19	
"are"	38	100M
"ing"	40	80M
"The"	37	50M
"cat"	901	2M
"runn"	1517	500k

vocabulary



"The cats are running" → [The, cat, s, are, runn, ing] → [37, 901, 19, 38, 1517, 40]

✅ **Advantages:** Manageable vocabulary + preserved structure + semantic proximity

Tokenizers Example

The tokenization of uncommon words like 'anticonstitutionnellement' and émojis 🤖 varies across tokenizers!

GPT2 Vocabulary size: **50,257**

Token count: **28**

Tokens:

THE	TOKEN	IZATION	OF	UNCOMMON	WORDS	LIKE	'	ANT	ICON	ST	ITUTION	NEL	LEMENT	'
AND	É	MO	J	IS			🤖	VARIES	ACROSS	TOKEN	IZERS	!		

Token IDs:

464	11241	1634	286	19185	2456	588	705	415	4749	301	2738	4954	1732	6	290
38251	5908	73	271	12520	97	244	17806	1973	11241	11341	0				

GPT5 Vocabulary size: **200,006**

Token count: **25**

Tokens:

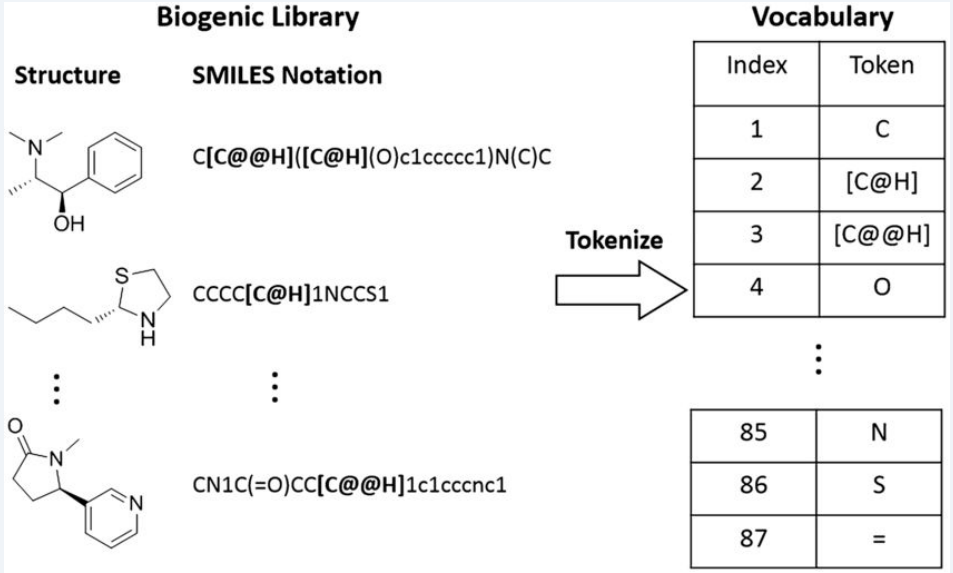
THE	TOKEN	IZATION	OF	UNCOMMON	WORDS	LIKE	'	ANT	ICON	STITUTION	NEL	LEMENT	'	
AND	É	MO	JIS		🤖	VARIES	ACROSS	TOKEN	IZERS	!				

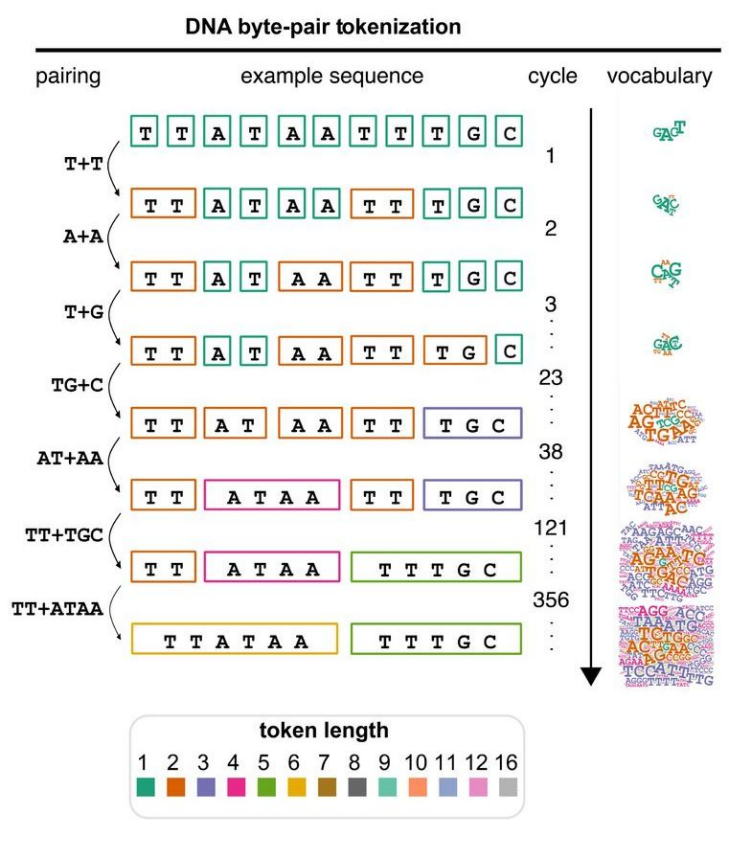
Token IDs:

976	6602	2860	328	69794	6391	1299	461	493	3679	20066	10085	1254	6	326	1212
3690	137286	93643	244	47245	5251	6602	24223	0							

scai
SORBONNE CLUSTER FOR ARTIFICIAL INTELLIGENCE

Text Data - Extended Sequence Application





Text Analysis

Application

Text Cleaning

Regex Cleaning

Remove URLs, emails, HTML tags

<P>Dude!!, this is such a great data set, wow.</P>
<P>I changed a couple column's modeling types and added a couple of new scripts to the table.</P>
<P></P>
<P> I tried modeling the data with a Response Surface Method (RSM). Rep(order?) seems to have an effect, so in model 2 I made that a Random Effect.</P>
<P></P>
<P> </P>
<P></P>
<P> </P>
<P>I'm not sure what you were hoping to find, but this is very interpretable data, and the study design is just fantastic too.</P>
<PRE><CODE class=" language-jsl">
Fit Model(
 Y(:Disease),
 Effects(
  :Name, :"Assessment time (DAI)"n & RS, :Plant type,
  :Name * :"Assessment time (DAI)"n,
  :"Assessment time (DAI)"n * :"Assessment time (DAI)"n, :Name * :Plant type,
  :"Assessment time (DAI)"n * :Plant type
 ),
 Random Effects(:"Rep (Order)"n),
 Personality("Standard Least Squares"),
 Emphasis("Effect Leverage"),
 Method("REML")
);</CODE></PRE>
<P>I changed the DAI column to numeric continuous, and the Rep Column to Numeric Continuous.</P>

Dude this is such a great data set wow I changed a couple column's modeling types and added a couple of new scripts to the table I tried modeling the data with a Response Surface Method RSM Rep order seems to have an effect so in model I made that a Random Effect I'm not sure what you were hoping to find but this is very interpretable data and the study design is just fantastic too I changed the DAI column to numeric continuous and the Rep Column to Numeric Continuous

Text Preprocessing - reduces vocabulary size

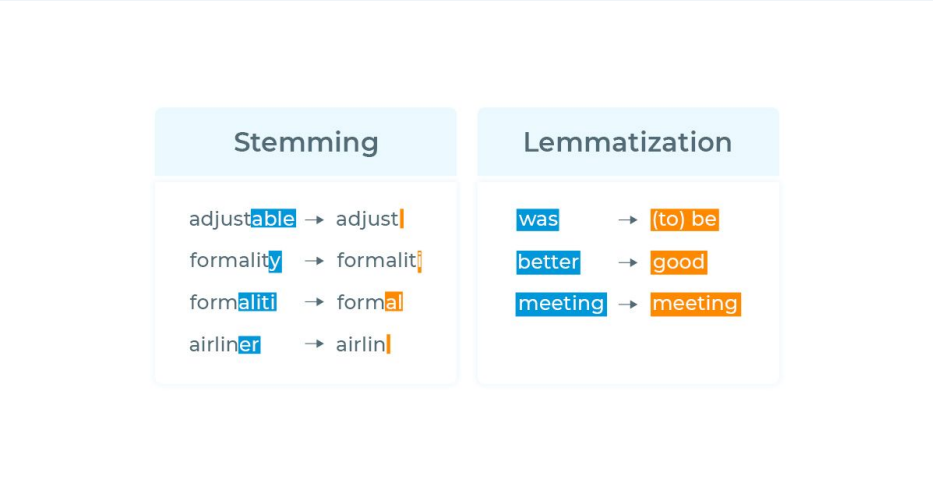
Common Preprocessing Steps

Lowercasing
"The Cat" ⟶ "the cat"

Punctuation Removal
"Hello, world!" ⟶ "Hello world"

Stopwords Removal
Remove functional words: "the", "is", "de", "le"

Stemming vs Lemmatization
Stemming (rule-based): "running" ⟶ "runn"
Lemmatization (dict-based): "running" ⟶ "run"

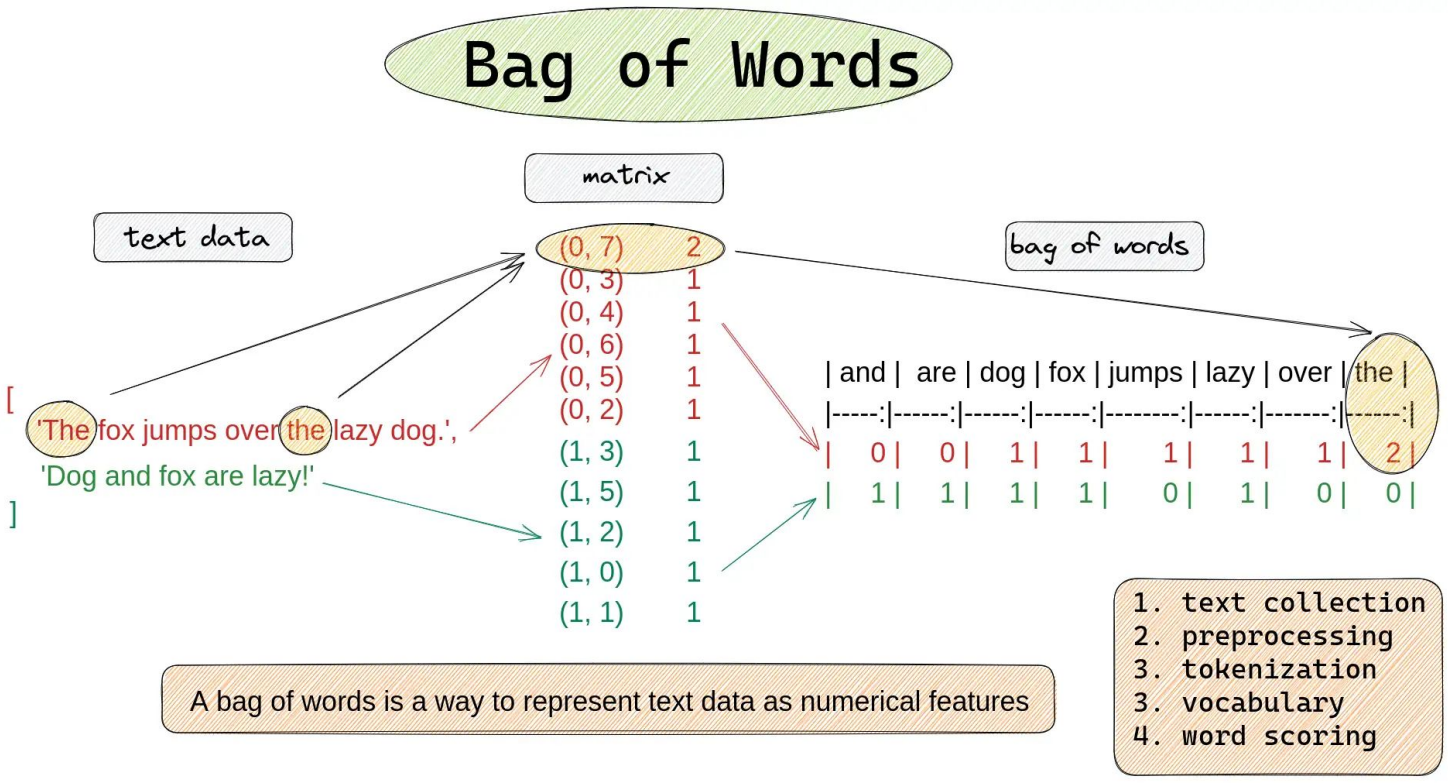


N-Gram

N-**gram** : contiguous sequence of *n* tokens from a text (unigram = 1, bigram = 2, trigram = 3, etc.), used to capture local word co-occurrence patterns

1-Gram	2-Gram	3-Gram
The	The Margherita	The Margherita pizza
Margherita	Margherita pizza	Margherita pizza is
pizza	pizza is	pizza is not
is	is not	is not bad
not	not bad	not bad taste
bad	bad taste	
taste		

Bag of Words (BoW)



TF-IDF : Term Frequency - Inverse Document Frequency

$\mathtt{tf}(t, d)$

	blue	bright	can	see	shining	sky	sun	today
1	1/2	0	0	0	0	1/2	0	0
2	0	1/3	0	0	0	0	1/3	1/3
3	0	1/3	0	0	0	1/3	1/3	0
4	0	1/6	1/6	1/6	1/6	0	1/3	0

x

$\mathtt{idf}(t, D)$

	blue	bright	can	see	shining	sky	sun	today
	0.602	0.125	0.602	0.602	0.602	0.301	0.125	0.602

- TF-IDF: Multiply TF and IDF scores, use to rank importance of words within documents
  - Most important word for each document is highlighted

$\mathtt{tfidf}(t, d, D) = \mathtt{tf}(t, d) \cdot \mathtt{idf}(t, D)$

	blue	bright	can	see	shining	sky	sun	today
1	**0.301**	0	0	0	0	0.151	0	0
2	0	0.0417	0	0	0	0	0.0417	**0.201**
3	0	0.0417	0	0	0	**0.100**	0.0417	0
4	0	0.0209	**0.100**	**0.100**	**0.100**	0	0.0417	0

BoW vs TF-IDF

	Bag of Words	TF-IDF
Values	Raw word counts	Weighted frequencies
Common words	High values (no penalty)	Low values (penalized by IDF)
Rare words	Low values	High values (rewarded)
Representation	Sparse	Sparse
Word order	Lost	Lost
Semantics	None	None

Both methods ignore word order and semantics → need dense representations (embeddings) to capture meaning

Quiz

1. Si on vous demande d'entraîner un modèle à classer si le sentiment est positif ou négatif sur des avis, comment faites-vous?

From Tokens to embeddings

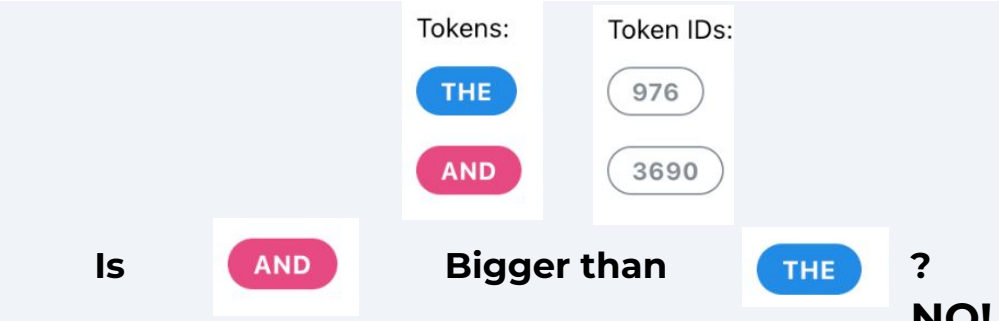
Breaking the token order

One hot encoding

Transforms a categorical variable into a binary vector.

One-hot vectors are sparse

No semantic similarity ("cat" and "dog" are equally distant from "car")



Token IDs are arbitrary assignments, not meaningful rankings

Vocabulary:
Man, woman, boy, girl, prince, princess, queen, king, monarch

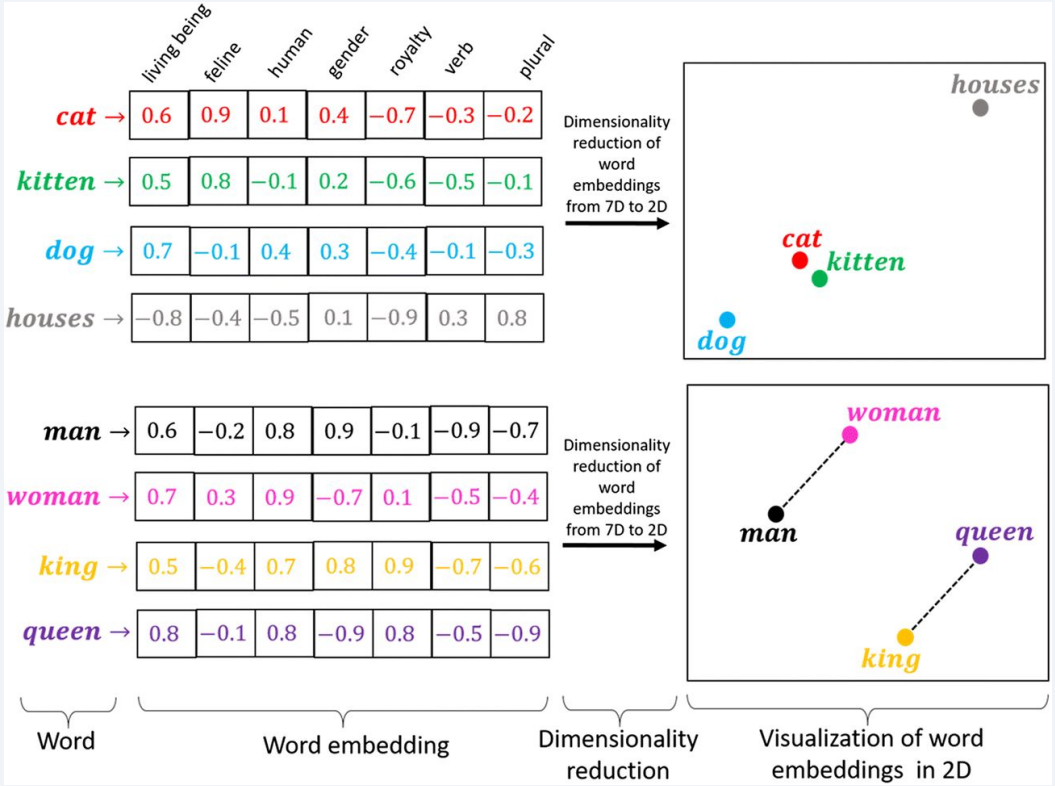
	1	2	3	4	5	6	7	8	9
man	1	0	0	0	0	0	0	0	0
woman	0	1	0	0	0	0	0	0	0
boy	0	0	1	0	0	0	0	0	0
girl	0	0	0	1	0	0	0	0	0
prince	0	0	0	0	1	0	0	0	0
princess	0	0	0	0	0	1	0	0	0
queen	0	0	0	0	0	0	1	0	0
king	0	0	0	0	0	0	0	1	0
monarch	0	0	0	0	0	0	0	0	1

Each word gets a 1x9 vector representation

Embeddings

Embedding

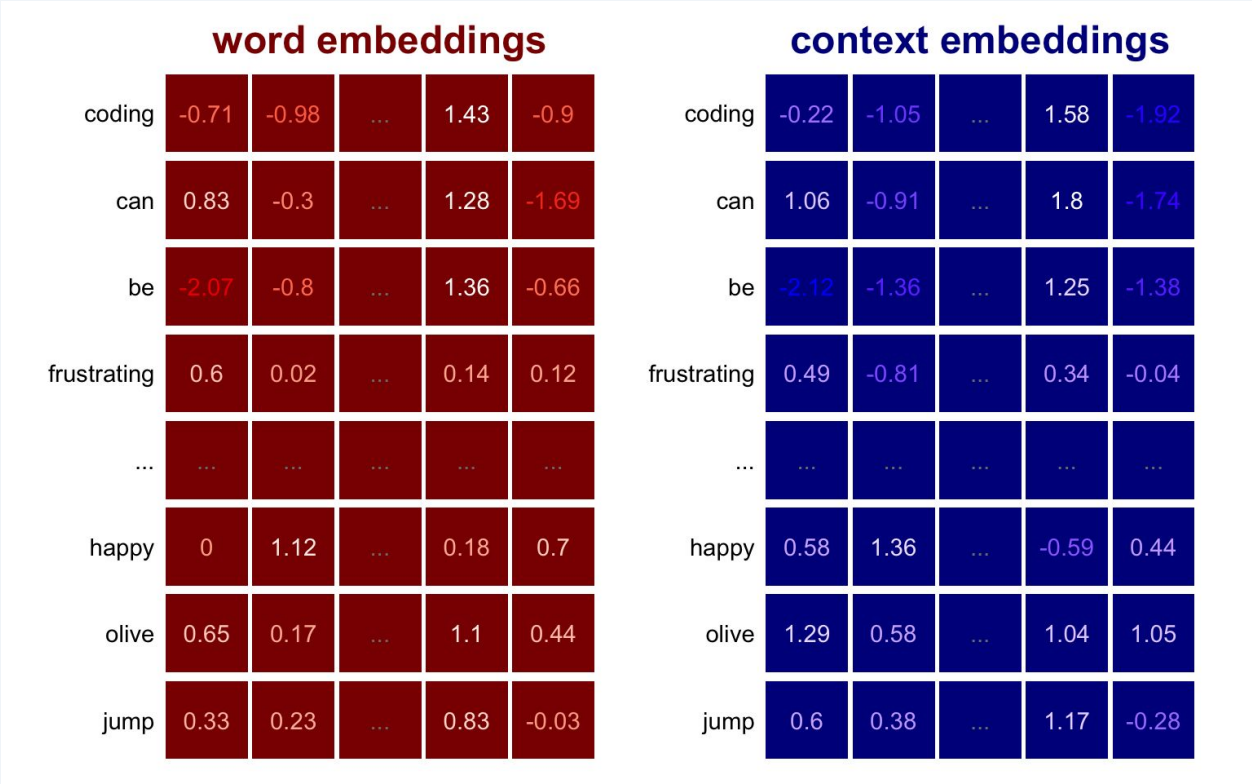
Learned mapping of a discrete object into a dense, low-dimensional continuous vector space where geometric proximity encodes semantic or structural similarity.



Embeddings

Word embedding: fixed vector per token

Context embedding: a vector per token that changes based on the full input sequence (e.g., "bank" gets different representations in "river bank" vs. "bank account").



From Linear to Non-Linear

Logistic Regression -> Classification

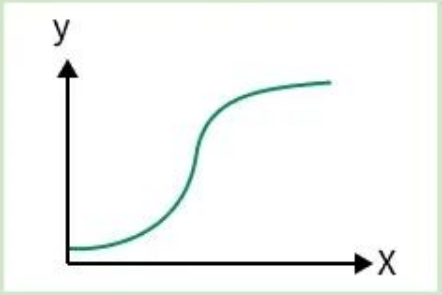
$$P(Y = 1 \mid x) = \sigma(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

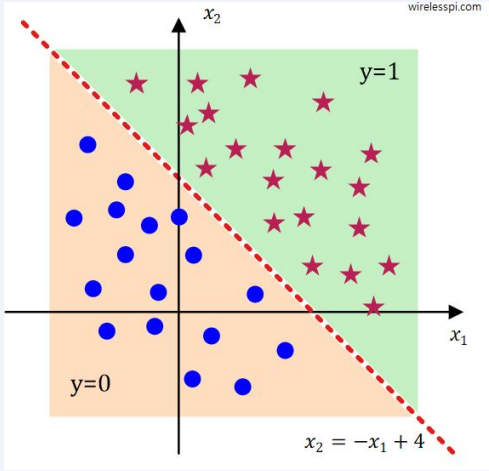
$$\hat{y} = \{1 \text{ si } \theta^T x > 0 \; 0 \text{ si } \theta^T x < 0\}$$

$$\hat{y} = \{1 \text{ si } P(Y=1|x) \geq 0.5 \; 0 \text{ sinon}\}$$

Logistic Regression

- Predicts categorical classes
- Uses sigmoid S-curve
- Solves classification problems

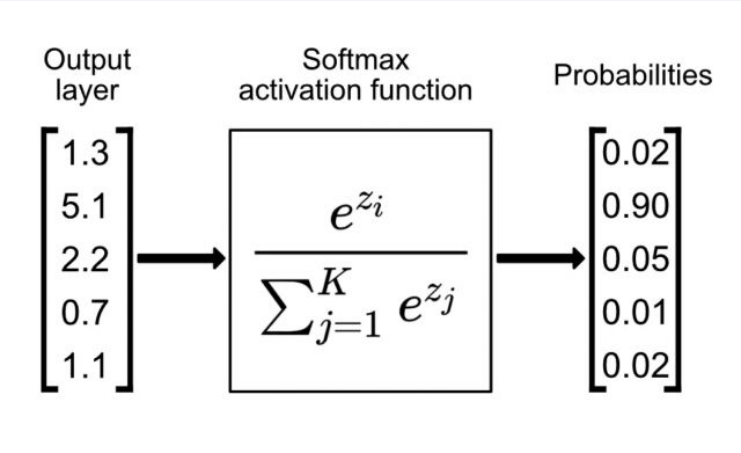


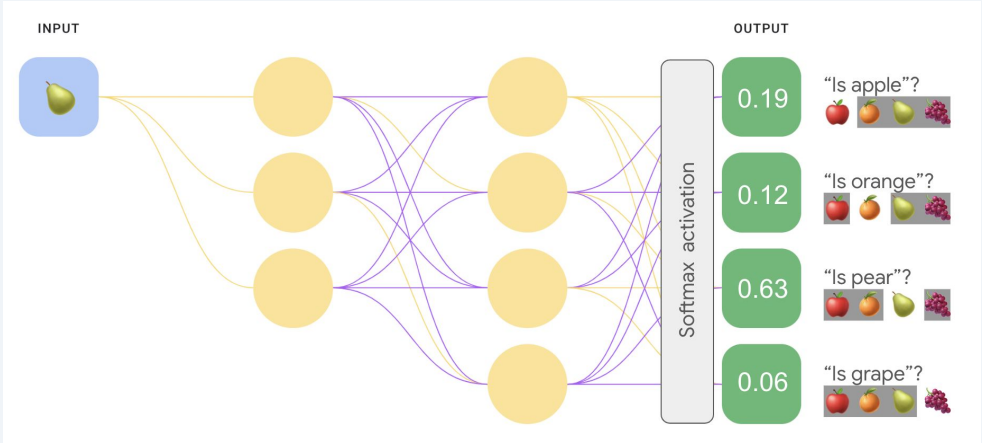


MLP Classification MultiClass

$$D=(X,Y)=(x_i,y_i)_{i\in\mathbb{N}}$$

$$\arg\min_{f\in\mathcal{F}} \ell(Y, f(X))$$

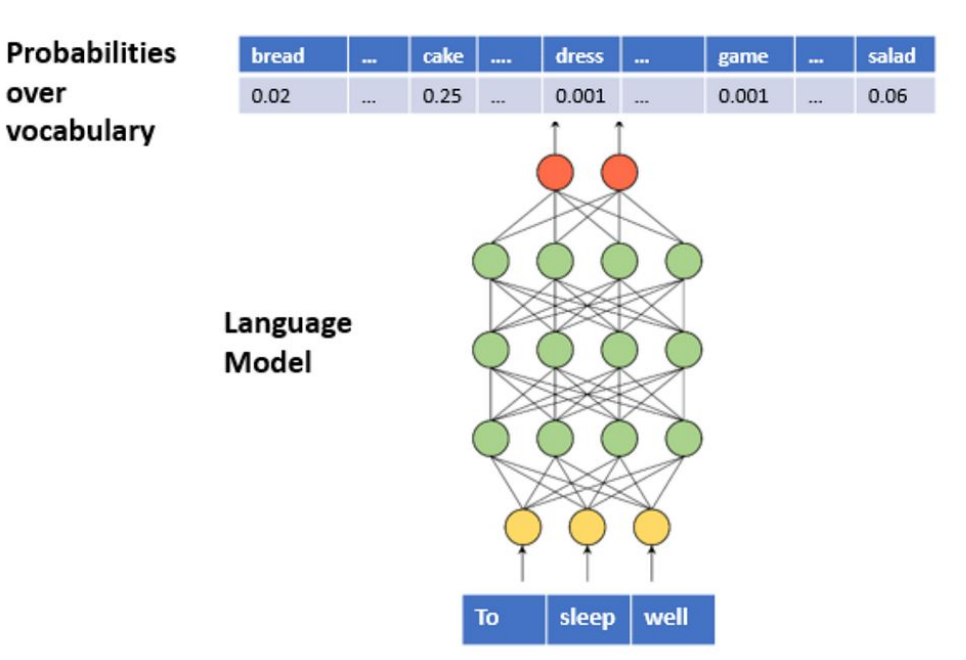




In **multiclass classification**, the model's final layer outputs one raw logit per class, and **softmax** ($\sigma(z_i) = e^{z_i} / \Sigma_j e^{z_j}$) converts these logits into a probability distribution that sums to 1, with the predicted class being the argmax; training typically minimizes **cross-entropy loss** between this distribution and the one-hot target.

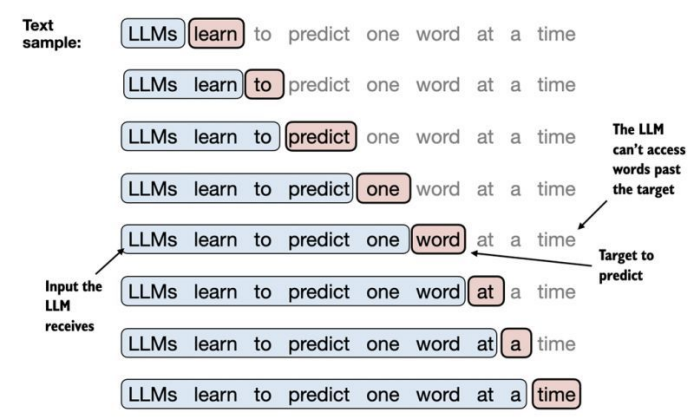
Generative Model

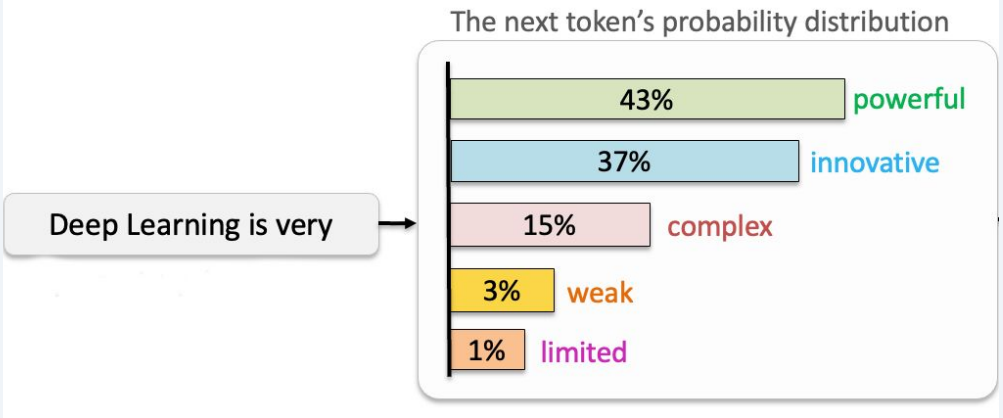
Generative Tasks



Autoregressive models (like gpt) are "classifiers" that, at each step, predicts a probability distribution over the vocabulary

Generative Tasks





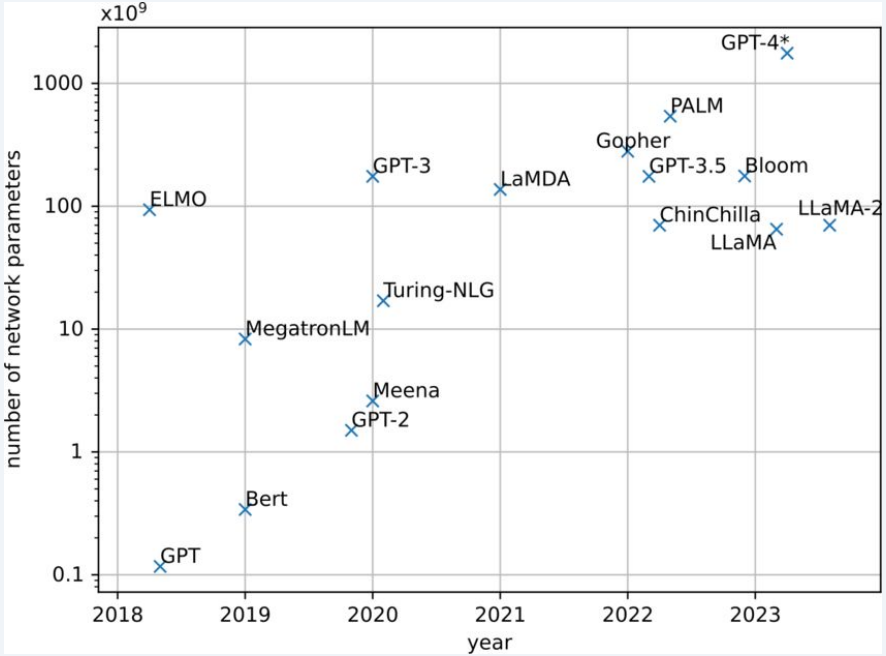
Training is exactly the same as other models. X is the sequence of tokens, Y the next token.

$$D=(X,Y)=(x_i,y_i)_{i\in\mathbb{N}} \quad \ell_{CE}(y,\hat{y}) = -\sum_{i=1}^{n}\sum_{c=1}^{C} y_{ic}\log(\hat{y}_{ic}) \quad \arg\min_{f\in\mathcal{F}} \ell(Y, f(X))$$

LLM

Large language Model

The LLM era began when models exceeded **1 billion parameters** and were trained on hundreds of billions to trillions of tokens — which places the start around GPT-2/GPT-3 (2019-2020).

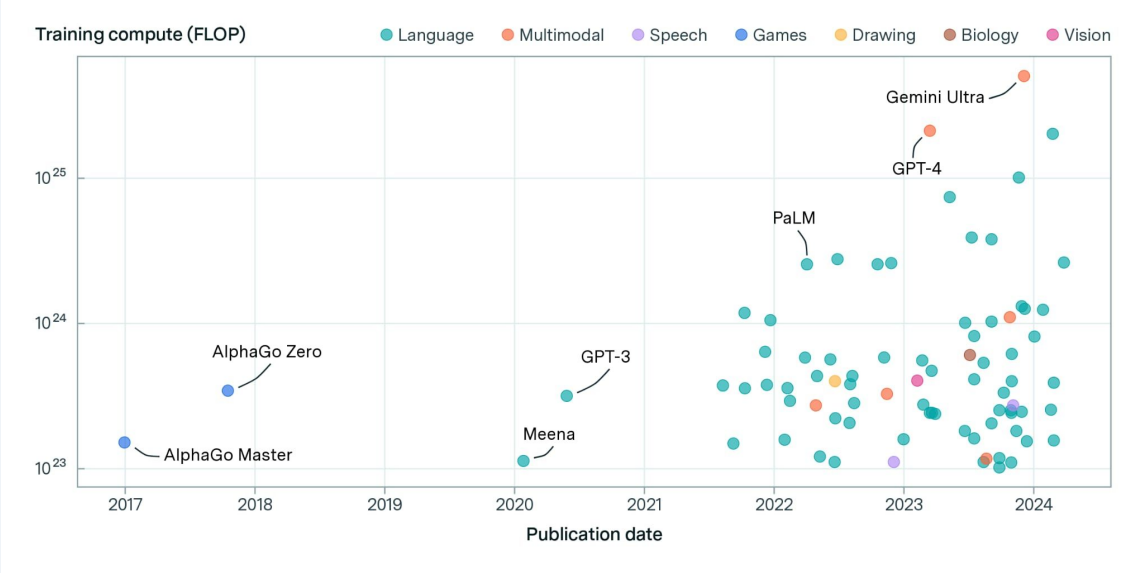


Number of parameters

	Training Set (Words)	**Training Set (Tokens)**
Recent LLMs		
Llama 3	11 trillion	15T
GPT-4	5 trillion	6.5T
Humans		
Human, age 5	30 million	
Human, age 20	150 million	

Data size

Training Large Model

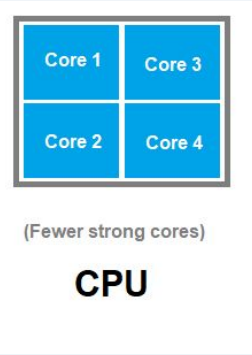


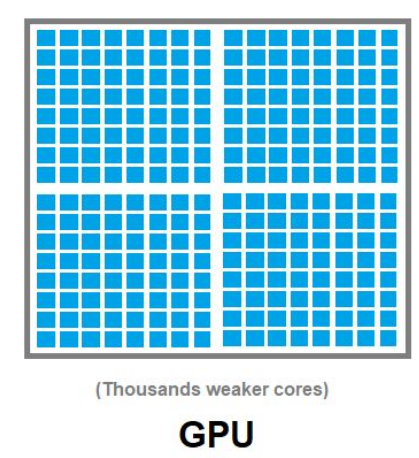
Training LLaMA 3.1 405B: 3.8×10^25 FLOPs

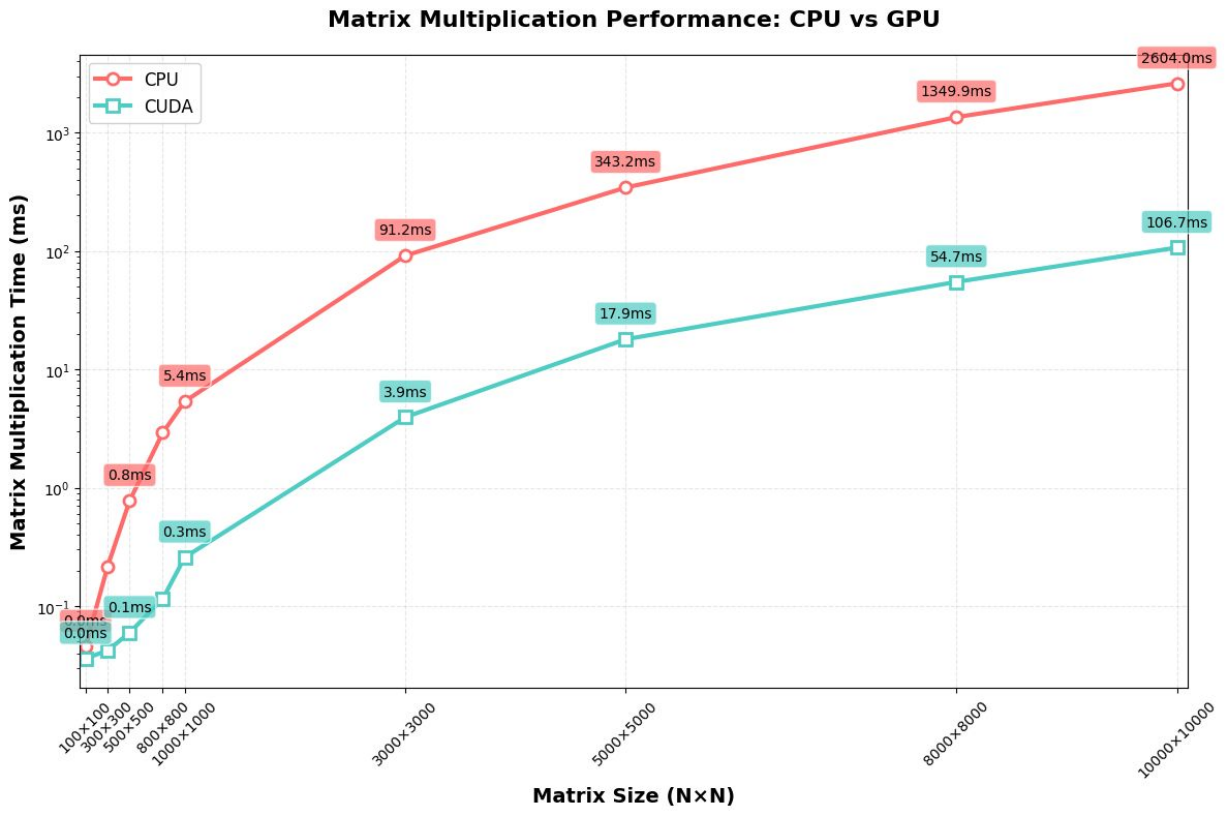
On a Personal Computer (200 GFLOPS)

Time = Total FLOPs / FLOPS = 1.9×10^{14} seconds = **6.0 million years**

CPU VS GPU


Core 1
Core 3
Core 2
Core 4
(Fewer strong cores)
CPU


(Thousands weaker cores)
GPU


Matrix Multiplication Performance: CPU vs GPU
CPU
CUDA
Matrix Multiplication Time (ms)
Matrix Size (N×N)
100×100
300×300
500×500
800×800
1000×1000
3000×3000
5000×5000
8000×8000
10000×10000
0.0ms
0.1ms
0.3ms
0.8ms
5.4ms
91.2ms
343.2ms
1349.9ms
2604.0ms
0.0ms
3.9ms
17.9ms
54.7ms
106.7ms

LLM : Resources and costs

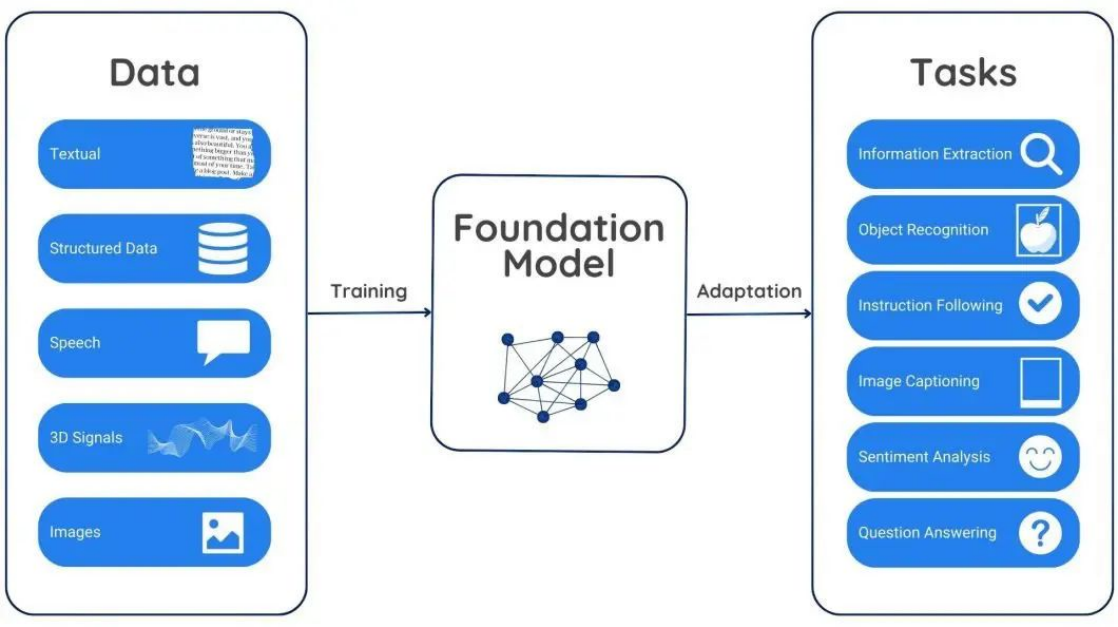
Parameters	Number GPU	Training Time	Estimated Cost
1B	~10-50 GPUs	Days to week	$10K - $100K
10B	~100-500 GPUs	Weeks to Month	$100K - $1M
100B+	1,000+ GPUs	Months	$1M - $100M+

Quiz

1. How chat-gpt is working?

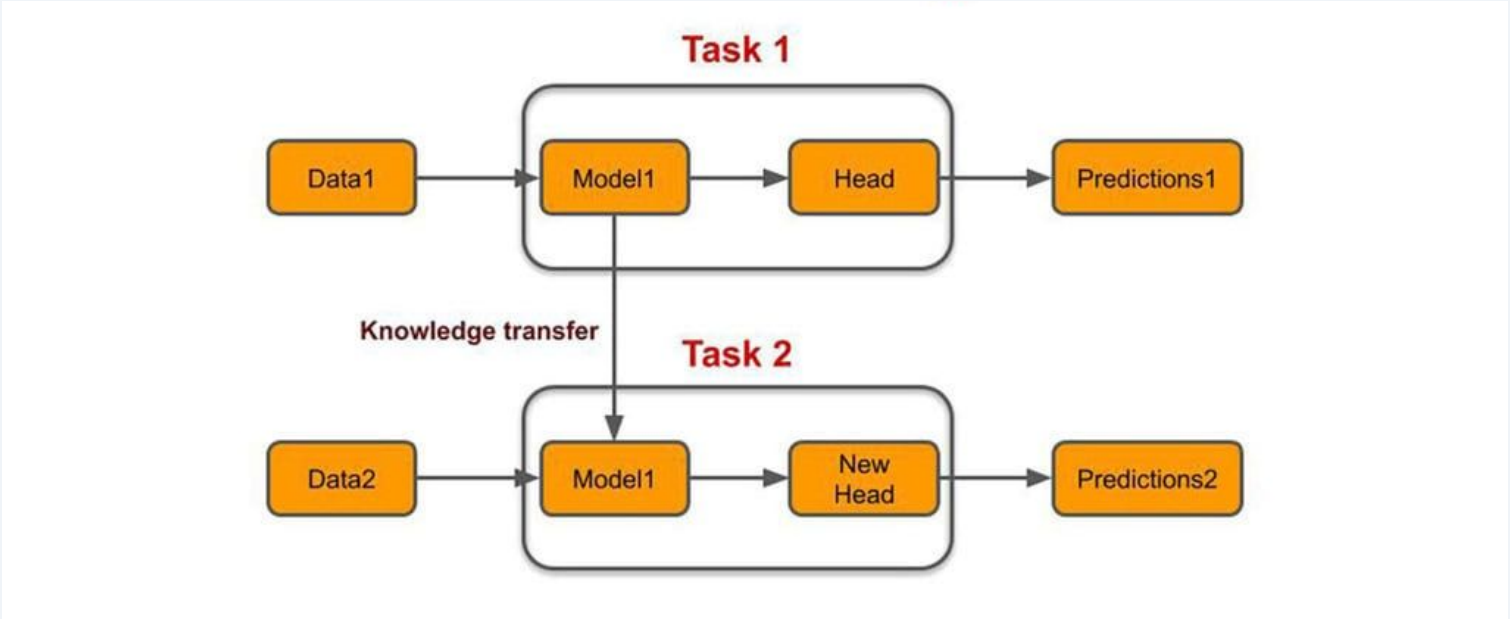
Transfer Learning pretrained Models

Foundation Model



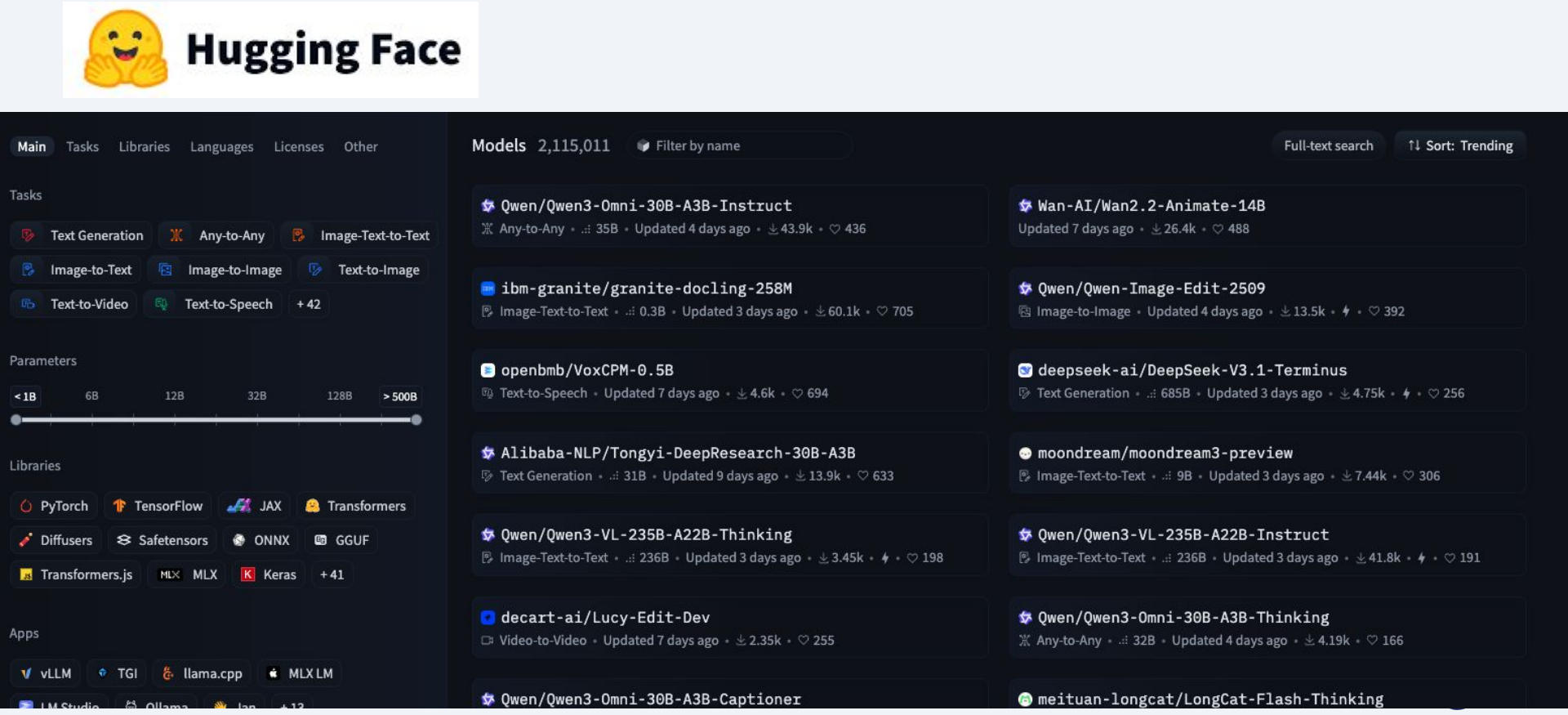
Foundation models are large AI models pre-trained on vast amounts of diverse data that can be adapted for a wide range of downstream tasks without training from scratch.

Transfer Learning



Instead of training a model from scratch (starting with randomly initialized parameters), we can start with a pre-trained model (one that has fine-tuned its parameters efficiently on a large dataset) and then fine-tune it in just a few iterations on our own (smaller) data.

Open Source Pre-trained Models



Data Science Librairies

scai

SORBONNE CLUSTER FOR
ARTIFICIAL INTELLIGENCE